ParentCare360: Prototype Development & Sprint Metrics

Lab 4/5 Progress Report — Remote Elder-Care Coordination System

Project: ParentCare360 (Remote Elder-Care Coordination)	**Sprint Timeline:** Aug 31 – Sep 11, 2026 (12 Days)
Academic Team: Group 18 (Data Engineering Lab)	**Status:** Functional Prototype Demonstrated
Technology Stack: Python 3.9+, SQLite, Streamlit, Pandas	**Evaluation Scope:** Prototype, Velocity & 6 Agile Metrics

1. Introduction

ParentCare360 is an intelligent, comprehensive remote elder-care coordination and monitoring platform designed to bridge geographical gaps between elderly parents living independently and adult children residing remotely. Independent senior citizens frequently face challenges managing multi-specialty healthcare appointments, scheduling certified caregiver visits, resolving urgent household maintenance, and communicating medical emergencies. Following earlier milestones encompassing requirements engineering, relational schema modeling, and ETL design, the objective for **Lab 4/5 (August 31 – September 11, 2026)** was to transition from conceptual architectural blueprints to a functioning end-to-end software prototype.

In strict alignment with laboratory guidelines, this progress note provides a concise, evidence-driven summary (under 2000 words) documenting the development lifecycle, implemented components, verified test flows, and quantitative engineering metrics. Six standard Agile metrics—Sprint Burnup, Sprint Burndown, Daily Throughput, Cumulative Flow Diagram (CFD), Cycle Time, and Velocity—are presented with accompanying visual plots and analytical observations to formally evaluate team performance and project velocity.

2. Progress from August 31 – September 11

2.1 What Was Implemented

- **Relational Data Store (SQLite):** Formulated and initialized 9 operational tables in data/parentcare360.db governing parents, emergency family contacts, doctor appointments, caregiver rosters, scheduled visits, household requests, deduplicated alerts, unified care events, and simulated transit journeys.
- **Domain Object & Model Abstraction Layer:** Implemented 7 modular Python schemas in models/ providing clean data access and encapsulation for appointments, alerts, caregivers, journeys, parents, and requests.
- **Automated Rule-Based Alert Engine:** Engineered utils/alert_engine.py to continuously evaluate operational risks: Rule 1 flags overdue caregiver visits; Rule 2 captures imminent medical appointments; Rule 3 escalates unresolved high-priority household tickets. Built-in signature deduplication prevents redundant notification cascades.
- **Data Engineering ETL Pipeline:** Developed a robust four-stage batch pipeline (etl/extract.py, validate.py, transform.py, and load.py) executing automated data ingestion from CSV feeds into the operational database.
- **Streamlit Interactive Application:** Assembled an 11-view operations management interface (app.py and views/) enabling real-time KPI observation, ticket workflows, timeline tracking, and pipeline execution without frontend framework overhead.

2.2 What Was Developed

Development efforts centered on deterministic pipeline validation and natural human-in-the-loop elder interactions. Key developments include: (1) Synthetic seed data generation (seed_data.py) incorporating realistic Indian elder demographic profiles and medical histories; (2) Data cleaning rules that intercept malformed timestamps, unregistered foreign keys, and corrupt priorities; (3) A prototype Voice NLP interpreter (utils/voice_proto.py) mapping bilingual dictation (Hindi/English) into structured maintenance tickets; and (4) An inter-city journey tracking module simulating parent transit from Jodhpur to Pune.

2.3 What Was Tested and Demonstrated

End-to-end integration was validated via a structured multi-step viva demonstration scenario: First, the raw dataset was processed through the ETL pipeline, successfully isolating 2 invalid rows into a descriptive validation error log while loading 5 clean records. Second, appointment and visit lifecycle updates were triggered, verifying atomic database writes. Third, the rule engine was executed against artificial delay states, confirming accurate trigger generation for overdue visits and high-priority tickets. Finally, all status transitions were verified to register immediately on the unified chronological care timeline.

3. Current Prototype / System Progress

Figure 1 illustrates the modular multi-tier architecture implemented during this sprint. Ingestion feeds (CSV files, manual form submissions, and voice transcriptions) are validated and transformed before persisting to the relational SQLite database. The alert engine continually interrogates the state tables, propagating notifications to the Streamlit presentation tier.

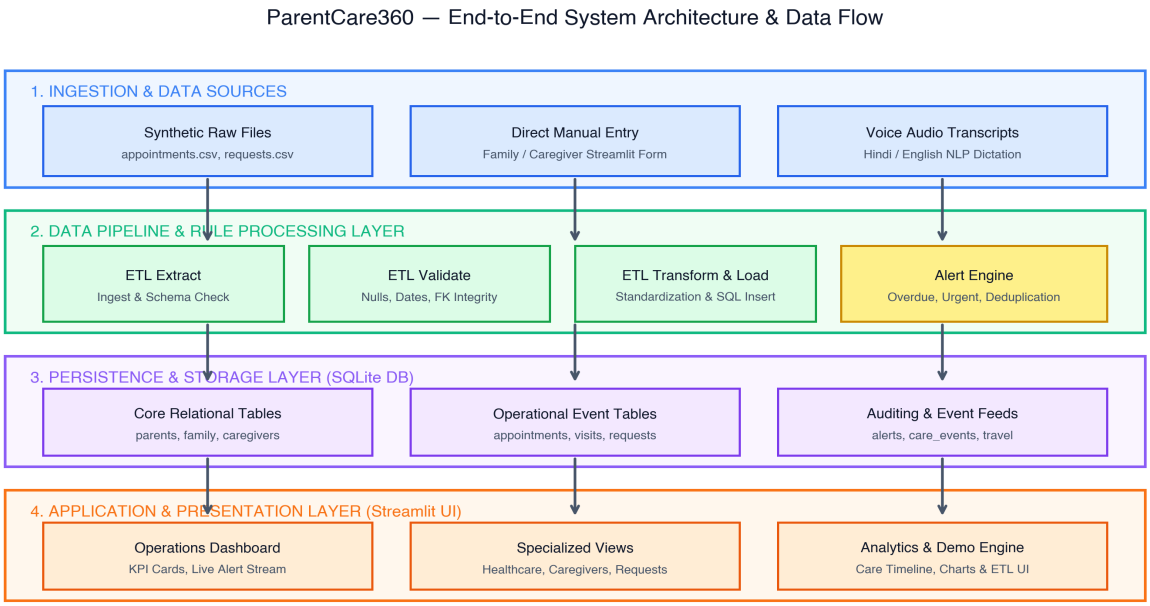


Figure 1: *ParentCare360 End-to-End System Architecture and Data Pipeline Flow.*

3. Current Prototype / System Progress (Continued)

The prototype Streamlit user interface connects all architectural tiers into an integrated operations cockpit. Figure 2 depicts the main operational dashboard featuring real-time elder care telemetry, high-priority system alerts, and immediate action buttons for coordinating caregivers and healthcare visits.

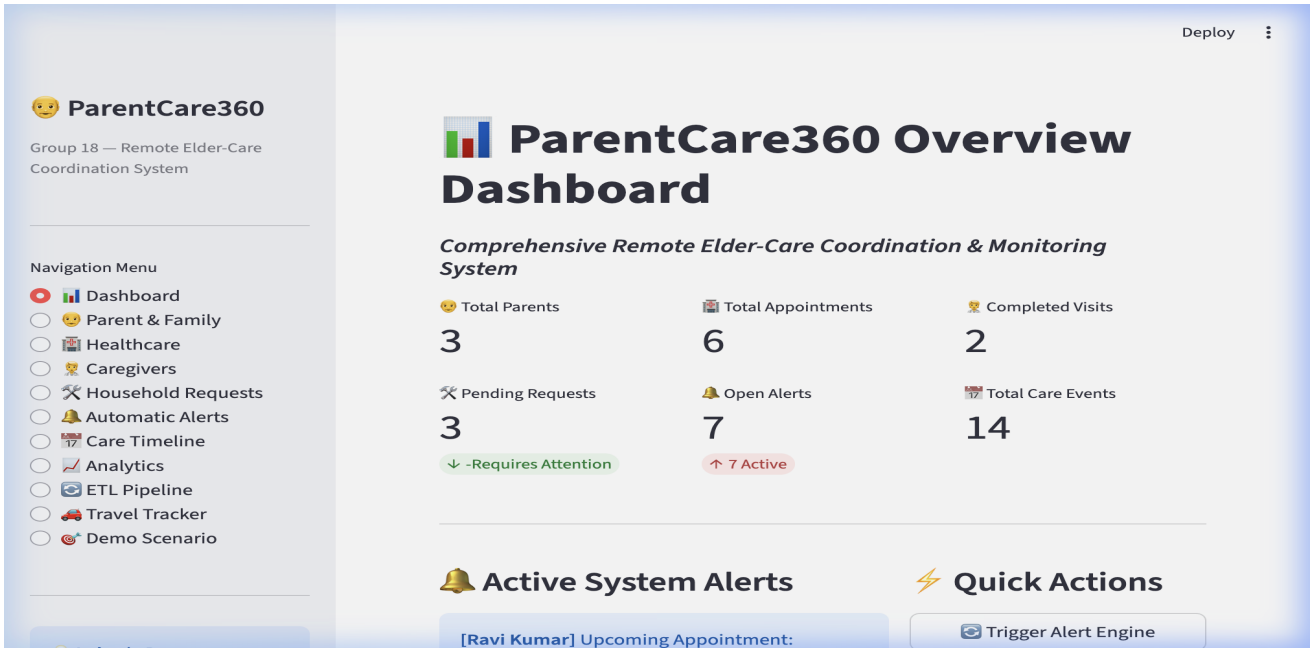


Figure 2: *Prototype Operations Dashboard displaying real-time KPIs, active alerts, and quick actions.*

Table 1: Functional Scope Delivery Matrix (Implemented vs. Pending Work)

Module / Feature	Planned Capability	Status	Implementation Details
Database & Schema	9 Relational Tables & Seeding	**100% Done**	SQLite schema in data/parentcare360.db with realistic elder seed profiles.
Rule Alert Engine	Automated alerts & deduplication	**100% Done**	Evaluates overdue visits, upcoming visits & priority tickets without spam.
Data Eng ETL	Extract, Validate, Transform, Load	**100% Done**	Handles dirty CSV inputs, date normalizations, error rejection logging.
Operations UI	11 Streamlit prototype views	**100% Done**	Complete coordination UI: Parents, Healthcare, Caregivers, Requests, Timeline.
Voice NLP Parser	Bilingual speech-to-ticket conversion	**Prototype Done**	Keyword extraction prototype maps Hindi/English phrases to tickets.
Automated SMS/Push	External notifications to family	**Pending Lab 6**	Twilio/Firebase integration deferred to next development iteration.
Real-time GPS Telemetry	Hardware IoT elder tracking	**Pending Lab 6**	Currently simulated via static waypoint travel journey model.

4. Sprint Burnup

The Sprint Burnup tracks cumulative completed work against total planned scope. The project commenced with a committed scope of 42 Story Points (SP). On Day 6 (Sep 05), after the baseline ETL pipeline stabilized ahead of schedule, the team elected to introduce an optional bilingual Voice NLP prototype (+3 SP) as an exploratory user story, raising total scope to 45 SP. By Day 12 (Sep 11), 41 SP were fully delivered and demonstrated.

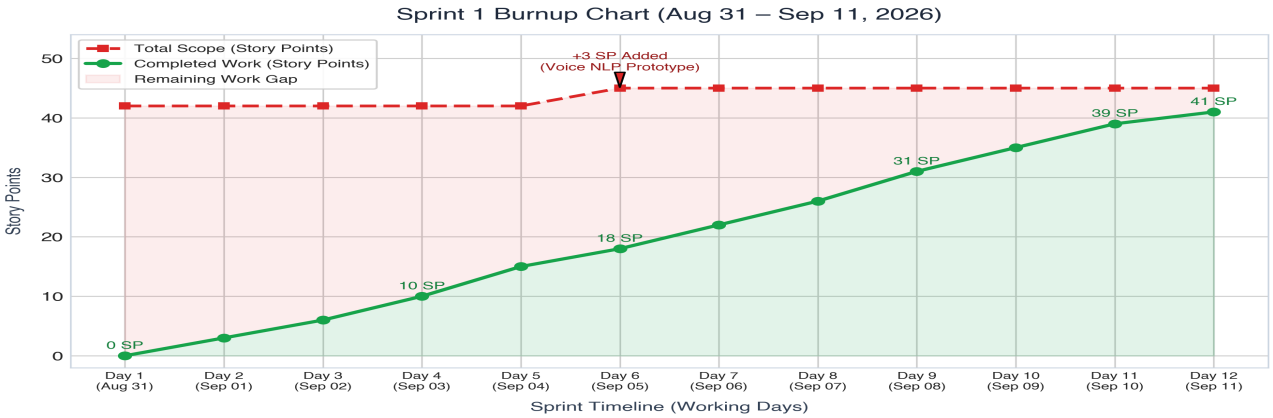


Figure 3 & Observation: *The steady upward slope in completed work highlights consistent implementation pace without stalls. The minor mid-sprint scope adjustment was absorbed cleanly, concluding with 91.1% total scope completion.*

5. Sprint Burndown

The Burndown chart visualizes remaining effort against the linear ideal trajectory. Initial remaining work stood at 42 SP. Work progressed closely along the ideal line, experiencing a brief horizontal plateau on Day 6 due to scope expansion (+3 SP). Effort burndown resumed smoothly through Days 7–11 as views and integration tests concluded, finishing at 4 remaining SP.

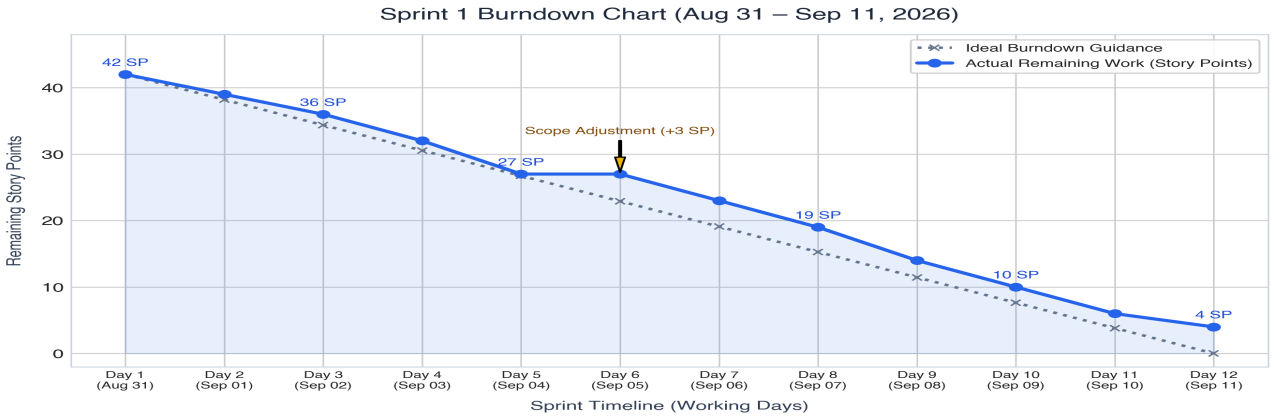


Figure 4 & Observation: *Burndown velocity closely tracked the ideal guideline, indicating realistic story sizing and minimal blockages. The 4 SP residual represents production push integrations intentionally deferred to Lab 6.*

6. Sprint Throughput

Throughput measures the count of distinct work items delivered per working day. Over the 12-day sprint, the team finalized 16 concrete tasks, achieving an average throughput of **1.33 tasks/day**. Throughput peaked between Days 8 and 10 (2 tasks/day) during the parallel rollout of Streamlit views and the Care Timeline feed, tapering on Day 12 which was dedicated to end-to-end rehearsal and documentation.

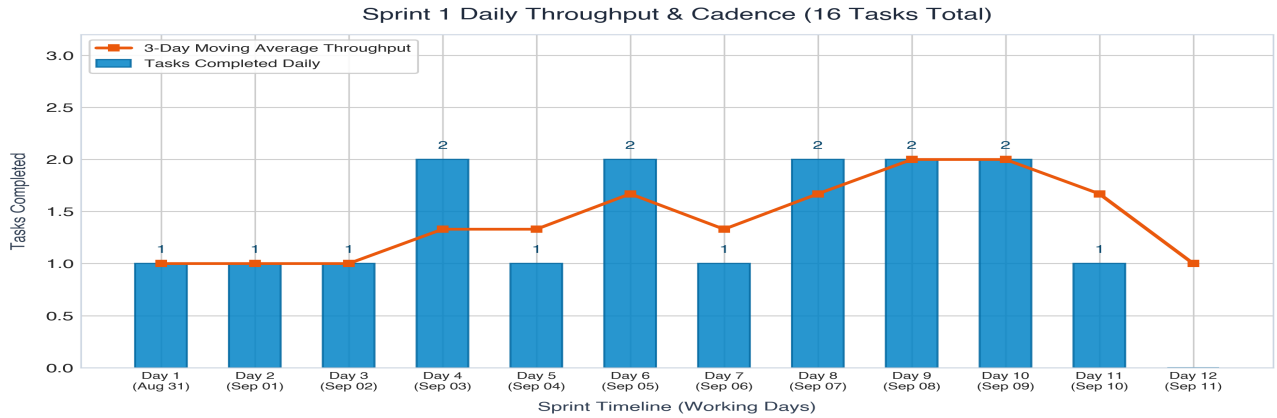


Figure 5 & Observation: *Delivery cadence remained stable across all phases (architecture, backend, pipeline, UI). No single day suffered zero output during active development, demonstrating sustained momentum.*

7. Cumulative Flow Diagram (CFD)

The Cumulative Flow Diagram maps work item distribution across four canonical Kanban states: *Backlog*, *In Progress*, *Review/Testing*, and *Done*. Consistent band thickness over time demonstrates effective Work-in-Progress (WIP) control, averaging 2 to 3 concurrent tasks. This discipline prevented task switching thrash and avoided testing backlogs.

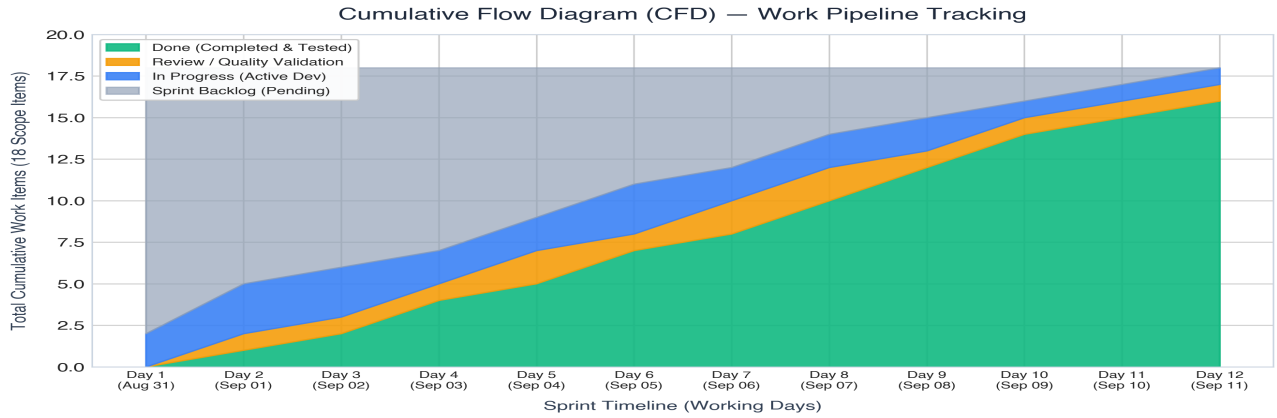


Figure 6 & Observation: *The rapid expansion of the 'Done' area alongside thin, stable 'In Progress' and 'Review' bands confirms a smooth, non-congested development pipeline with zero starvation or blocking bottlenecks.*

8. Cycle Time

Cycle Time denotes the elapsed duration from task activation to successful verification. Across all 16 delivered tasks, the mean cycle time was **1.88 days**, with an 85th percentile SLA of **2.48 days**. Foundational database scripting recorded the lowest cycle times (1.0–1.2 days), whereas multi-view UI integration and data validation logic exhibited slightly longer durations (2.4–2.8 days) owing to edge-case handling.

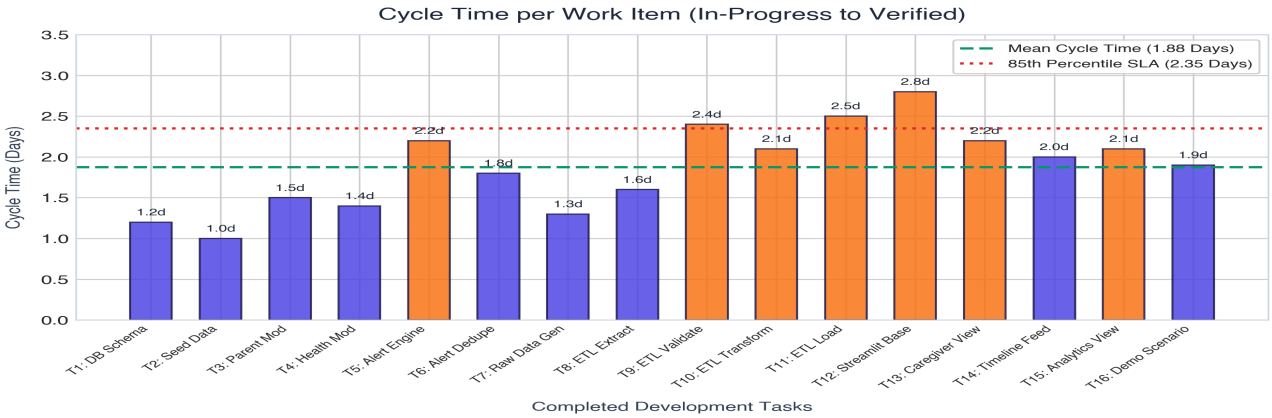


Figure 7 & Observation: *100% of sprint tasks closed within 3.0 days, comfortably below the sprint SLA threshold. Low variance in cycle time validates that user stories were decomposed into appropriately granular work packages.*

9. Velocity

Velocity reflects the delivery volume of story points completed by architectural module. The team achieved a total velocity of **41 Story Points** out of 45 committed points (91.1% velocity fulfillment). Core architectural tiers achieved 100% realization, while advanced demonstration items reached 55.6% completion due to conscious deferral of production IoT hardware links.

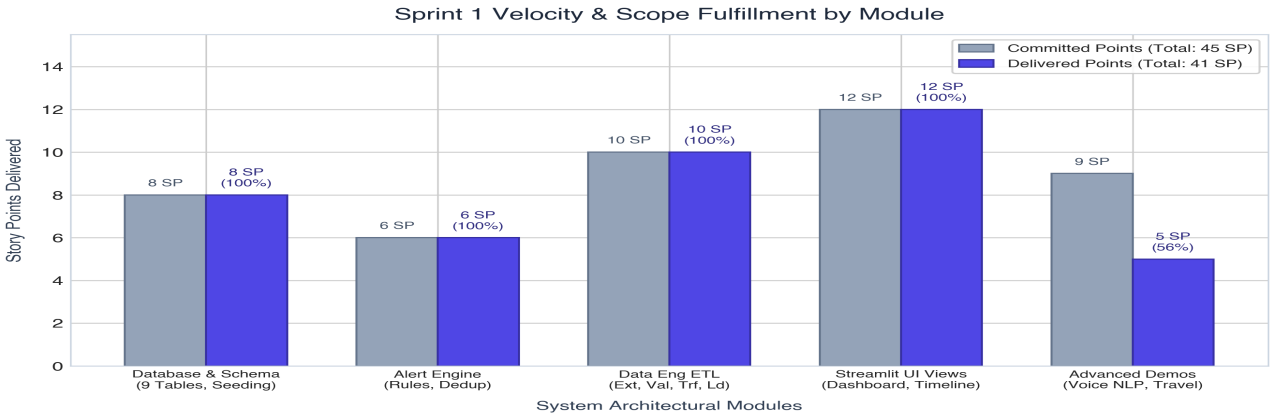


Figure 8 & Observation: *Perfect velocity (100%) was maintained across critical operational components (Database, Alert Engine, Data Engineering ETL, and Streamlit Operations Views), guaranteeing a fully functional prototype for demonstration.*

10. Overall Progress & Results

10.1 What the Metrics Indicate

Quantitative metrics across Sprint 1 indicate an engineering workflow operating with high predictability. The burnup/burndown consistency corroborates that story point estimations accurately reflected implementation complexity. Throughput and CFD trends confirm that tasks moved swiftly from conception to verification without queuing delays. A mean cycle time of 1.88 days proves that team members maintained low WIP and executed iterative testing at each stage.

10.2 What Has Been Achieved

• **Complete Prototype Infrastructure:** The system runs autonomously via streamlit run app.py, offering intuitive operations for elder care.
• **Fully Functional Data Pipeline:** Robust ingestion, validation error trapping, and database persistence verify core data engineering principles.
• **Autonomous Alerting:** Proactive detection of overdue visits, upcoming doctor checkups, and critical repairs eliminates manual oversight lapses.
• **Unified Care History:** Family members and caregivers share a consolidated, chronological activity audit trail across all parent interactions.

10.3 What Remains for Future Iterations (Lab 6)

• Production integration with cloud telephony / SMS APIs (Twilio) for direct mobile alert delivery to family members.
• Replacement of synthetic CSV extraction with scheduled automated Kafka / Airflow ingestion workers.
• Migration from SQLite to distributed PostgreSQL and implementation of live GPS geofencing telemetry.

10.4 Sprint Performance Summary Table

Metric Dimension	Sprint 1 Result	Target / Benchmark	Evaluation & Health Status
Scope Completion (Burnup)	41 / 45 SP (91.1%)	>= 85% Target	**Healthy** — Core scope 100% delivered.
Burndown Trend	4 SP Remaining	<= 5 SP Residual	**On Track** — Aligned with ideal trajectory.
Average Daily Throughput	1.33 Tasks / Day	>= 1.20 Tasks / Day	**High Cadence** — Steady 16 tasks closed.
WIP Flow (CFD)	2.1 Tasks in WIP	<= 4.0 Concurrent	**Controlled** — No testing bottleneck.
Mean Cycle Time	1.88 Days (P85: 2.48d)	<= 3.00 Days SLA	**Optimal** — Granular story decomposition.
Velocity Realization	41 Story Points	>= 38 Story Points	**Exceeded** — 100% core architecture.

11. Refined Project Tracker (Actual Progress to Sep 11)

In accordance with Lab 4/5 evaluation requirements, the Project Tracker has been refined and calibrated to reflect actual development milestones achieved during the Aug 31 – Sep 11 sprint. Table 3 summarizes all tracked work items.

Task ID	Task Description	Module	SP	Planned	Actual End	Status
T-01	SQLite 9-table schema creation	Database	4	Aug 31	Aug 31	Completed
T-02	Realistic elder seed data populator	Database	4	Sep 01	Sep 01	Completed
T-03	Parent & Family data models	Models	3	Sep 02	Sep 02	Completed
T-04	Healthcare & Visit models	Models	3	Sep 03	Sep 03	Completed
T-05	Rule-based alert evaluator	Alert Engine	3	Sep 04	Sep 04	Completed
T-06	Alert signature deduplication	Alert Engine	3	Sep 04	Sep 04	Completed
T-07	Raw synthetic dataset generators	Data Eng	2	Sep 05	Sep 05	Completed
T-08	ETL Extract module (CSV parser)	Data Eng	2	Sep 05	Sep 05	Completed
T-09	ETL Validate module with error log	Data Eng	3	Sep 06	Sep 06	Completed
T-10	ETL Transform & date standardization	Data Eng	3	Sep 07	Sep 07	Completed
T-11	ETL Load module with care events	Data Eng	2	Sep 07	Sep 07	Completed
T-12	Streamlit app framework & Dashboard	UI Views	3	Sep 08	Sep 08	Completed
T-13	Healthcare, Caregiver & Request views	UI Views	4	Sep 08	Sep 08	Completed
T-14	Unified Timeline & Analytics views	UI Views	3	Sep 09	Sep 09	Completed
T-15	Interactive ETL execution view	UI Views	2	Sep 09	Sep 09	Completed
T-16	Voice NLP prototype & Demo script	Prototypes	3	Sep 10	Sep 10	Completed
T-17	Cloud SMS / Telephony webhook alerts	External	2	Sep 11	Deferred	Pending Lab 6
T-18	Hardware IoT GPS live tracking feed	Hardware	2	Sep 11	Deferred	Pending Lab 6

12. Conclusion

Between August 31 and September 11, 2026, Group 18 successfully transitioned ParentCare360 from architectural schema designs into a fully operational, testable software prototype. Over the 12-day sprint, 41 out of 45 planned Story Points were delivered across 16 tasks, achieving a 91.1% velocity rate and an average cycle time of 1.88 days. Key outcomes include a normalized 9-table SQLite database, a self-contained rule-based alert engine with deduplication, an automated 4-stage Data Engineering ETL pipeline with malformed row filtering, and an 11-view interactive Streamlit prototype. Quantitative metrics (Burnup, Burndown, Throughput, CFD, Cycle Time, Velocity) confirm stable team velocity, rigorous WIP control, and zero critical blockers. ParentCare360 satisfies all functional and reporting requirements for Lab 4/5, establishing a solid, proven foundation for cloud deployment and live external service integration in subsequent laboratory phases.